

```

import React, {
  createContext,
  useContext,
  useState,
  useEffect,
  useRef,
  useCallback,
} from "react";
import { X } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🌀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// 🚪 LOCAL TYPE ISOLATION GATEWAY
export interface DrawerRootProps {
  open: boolean;
  onOpenChange: (open: boolean) => void;
  children: React.ReactNode;
}

export interface DrawerOverlayProps extends React.HTMLAttributes<HTMLDivElement> {
  className?: string;
}

export interface DrawerContentProps extends React.HTMLAttributes<HTMLDivElement> {
  size?: "sm" | "md" | "lg" | "xl" | "full";
}

export interface DrawerHeaderProps extends React.HTMLAttributes<HTMLDivElement> {
  title: string;
  description?: string;
}

interface DrawerContextValue {
  open: boolean;
  onOpenChange: (open: boolean) => void;
  size: DrawerContentProps["size"];
}

const DrawerContext = createContext<DrawerContextValue | null>(null);

function useDrawerContext() {
  const context = useContext(DrawerContext);
  if (!context) {
    throw new Error("Drawer components must be used within DrawerRoot");
  }
}

```

```

    return context;
}

const SIZE_CLASSES = {
  sm: "w-[320px] max-w-[90vw]",
  md: "w-[400px] max-w-[90vw]",
  lg: "w-[560px] max-w-[90vw]",
  xl: "w-[720px] max-w-[90vw]",
  full: "w-full max-w-[100vw]",
} as const;

export function DrawerRoot({ open, onOpenChange, children }: DrawerRootProps) {
  const [size] = useState<DrawerContentProps["size"]>("md");

  const handleOpenChange = useCallback(
    (newOpen: boolean) => {
      onOpenChange(newOpen);
    },
    [onOpenChange],
  );

  const value: DrawerContextValue = {
    open,
    onOpenChange: handleOpenChange,
    size,
  };

  return (
    <DrawerContext.Provider value={value}>
      {React.Children.map(children, (child) => {
        if (!React.isValidElement(child)) return child;
        if (child.type === DrawerContent) {
          const childProps = child.props as DrawerContentProps;
          // eslint-disable-next-line @typescript-eslint/no-explicit-any
          return React.cloneElement(child as any, {
            size: childProps.size || size,
          });
        }
        return child;
      })}
    </DrawerContext.Provider>
  );
}

export function DrawerOverlay({ className, ...props }: DrawerOverlayProps) {
  const { open, onOpenChange } = useDrawerContext();

  if (!open) return null;

  const handleClick = useCallback(() => {

```

```

    onOpenChange(false);
  }, [onOpenChange]);

return (
  <div
    className={cn(
      "fixed inset-0 z-40",
      "bg-black/50 backdrop-blur-sm",
      "transition-opacity duration-300 ease-out",
      open ? "opacity-100" : "opacity-0 pointer-events-none",
      className,
    )}
    onClick={handleClick}
    aria-hidden="true"
    {...props}
  />
);
}

export function DrawerContent({
  className,
  size = "md",
  children,
  ...props
}: DrawerContentProps) {
  const { open, onOpenChange } = useDrawerContext();
  const contentRef = useRef<HTMLDivElement>(null);
  const previousActiveElement = useRef<HTMLElement | null>(null);

  const handleKeyDown = useCallback(
    (event: KeyboardEvent) => {
      if (event.key === "Escape") {
        onOpenChange(false);
      }
    },
    [onOpenChange],
  );

  const handleFocusTrap = useCallback((event: KeyboardEvent) => {
    if (event.key !== "Tab" || !contentRef.current) return;

    const focusableElements = contentRef.current.querySelectorAll<HTMLElement>(
      'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])',
    );

    const firstElement = focusableElements[0];
    const lastElement = focusableElements[focusableElements.length - 1];

    if (event.shiftKey && document.activeElement === firstElement) {
      event.preventDefault();

```

```

    lastElement?.focus();
  } else if (!event.shiftKey && document.activeElement === lastElement) {
    event.preventDefault();
    firstElement?.focus();
  }
}, []));

useEffect(() => {
  if (open) {
    previousActiveElement.current = document.activeElement as HTMLElement;
    document.addEventListener("keydown", handleKeyDown);
    document.addEventListener("keydown", handleFocusTrap);
    document.body.style.overflow = "hidden";

    const focusableElement = contentRef.current?.querySelector<HTMLElement>(
      'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])',
    );
    focusableElement?.focus();

    return () => {
      document.removeEventListener("keydown", handleKeyDown);
      document.removeEventListener("keydown", handleFocusTrap);
      document.body.style.overflow = "";
      previousActiveElement.current?.focus();
    };
  }
}, [open, handleKeyDown, handleFocusTrap]);

if (!open) return null;

return (
  <div
    ref={contentRef}
    className={cn(
      "fixed right-0 top-0 z-50 h-full",
      "flex flex-col",
      "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
      "border-1 border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
      "shadow-[var(--shadow-xl)]",
      "transition-transform duration-300 ease-out",
      open ? "translate-x-0" : "translate-x-full",
      SIZE_CLASSES[size],
      className,
    )}
    role="dialog"
    aria-modal="true"
    aria-label="Drawer"
    {...props}
  >
    {children}
  </div>

```



```

    type="button"
    onClick={handleClick}
    className={cn(
      "flex-shrink-0 p-1.5 rounded-md",
      "text-muted-foreground hover:text-foreground",
      "hover:bg-[color-mix(in_oklch,var(--color-muted)_50%,transparent)]",
      "active:scale-95",
      "transition-all duration-150 ease-out",
      "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 focus-visible:ring-offset-background",
      className,
    )}
    aria-label="Close drawer"
    {...props}
  >
    <X size={18} strokeWidth={2.5} aria-hidden="true" />
  </button>
);
}

export default DrawerRoot;

```